EXPERIMENT -5

AIM- Create SQL views for department salary summary and employee hierarchy. Test updatability of views. Implement a recursive CTE to display reporting chains.

OBJECTIVES

1. To understand the concept and purpose of SQL views.
2. To create a view for displaying department-wise salary summaries.
3. To create a view representing the employee-manager hierarchy.
4. To test whether the created views are updatable.
5. To understand the limitations of updating views containing aggregate functions and joins.
6. To understand Common Table Expressions (CTEs).
7. To implement a recursive CTE for traversing employee-manager relationships.
8. To display complete reporting chains from employees to the top-level manager.

THEORY

SQL View

A **view** is a virtual table created from the result of a SQL query. It does not normally store the actual data separately; instead, it stores the query definition used to retrieve the data from underlying tables.

Views are useful for:

- Simplifying complex queries.
- Providing a customized representation of data.
- Restricting access to specific columns or rows.
- Improving data abstraction.
- Reusing frequently required queries.

Department Salary Summary View

A department salary summary view combines employee and department information and uses aggregate functions such as COUNT() and AVG() to provide summarized salary information for each department.
For example, it can display:

- Department name
- Number of employees

- Total salary
- Average salary
- Maximum salary
- Minimum salary

Because this view contains **aggregate functions and GROUP BY**, it is generally not directly updatable.

Employee Hierarchy View

An employee hierarchy represents the relationship between an employee and their manager.

For this purpose, the Employee table contains a ManagerID attribute that refers back to EmpID in the same table.

Updatable View

An updatable view is a view through which operations such as INSERT, UPDATE, or DELETE can be performed on the underlying table.

A simple view based on a single table is generally more likely to be updatable.

Non-Updatable Views

A view may not be directly updatable when its definition contains features that make it difficult to map the result back to individual base-table rows.

Common examples include views containing:

- Aggregate functions
- GROUP BY
- DISTINCT
- Set operations
- Certain joins
- Calculated or derived results

Therefore, a department salary summary containing SUM(), AVG(), and GROUP BY cannot normally be directly updated.
Similarly, the employee hierarchy view uses a self-join, so it should not be treated as a simple updatable view.

Common Table Expression

A **Common Table Expression (CTE)** is a temporary named result set that can be referenced within a SQL statement.

It is defined using the WITH clause.

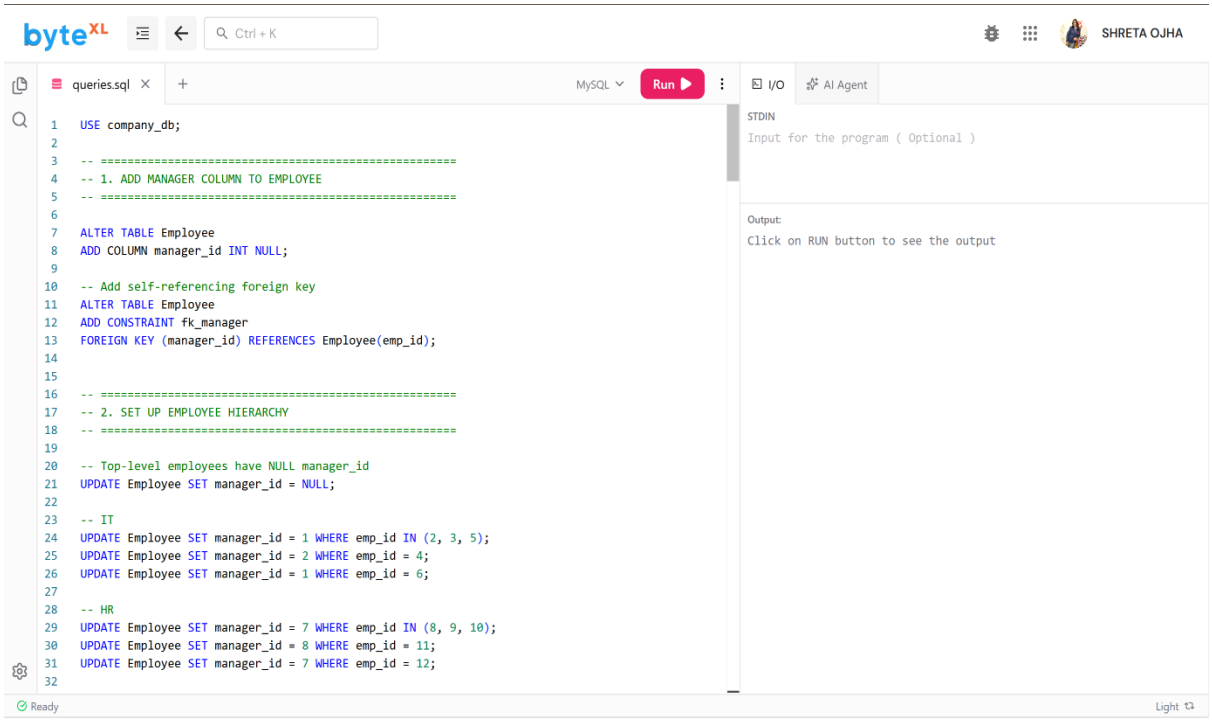
Recursive CTE

A **recursive CTE** is a CTE that refers to itself. It is useful for processing hierarchical or tree-structured data.

A recursive CTE normally consists of two parts:

1. **Anchor member**: identifies the starting records.
2. **Recursive member**: repeatedly finds the next level of related records.

SOURCE CODE




```sql
USE company_db;

-- ======================================================
-- 1. ADD MANAGER COLUMN TO EMPLOYEE
-- ======================================================

ALTER TABLE Employee
ADD COLUMN manager_id INT NULL;

-- Add self-referencing foreign key
ALTER TABLE Employee
ADD CONSTRAINT fk_manager
FOREIGN KEY (manager_id) REFERENCES Employee(emp_id);

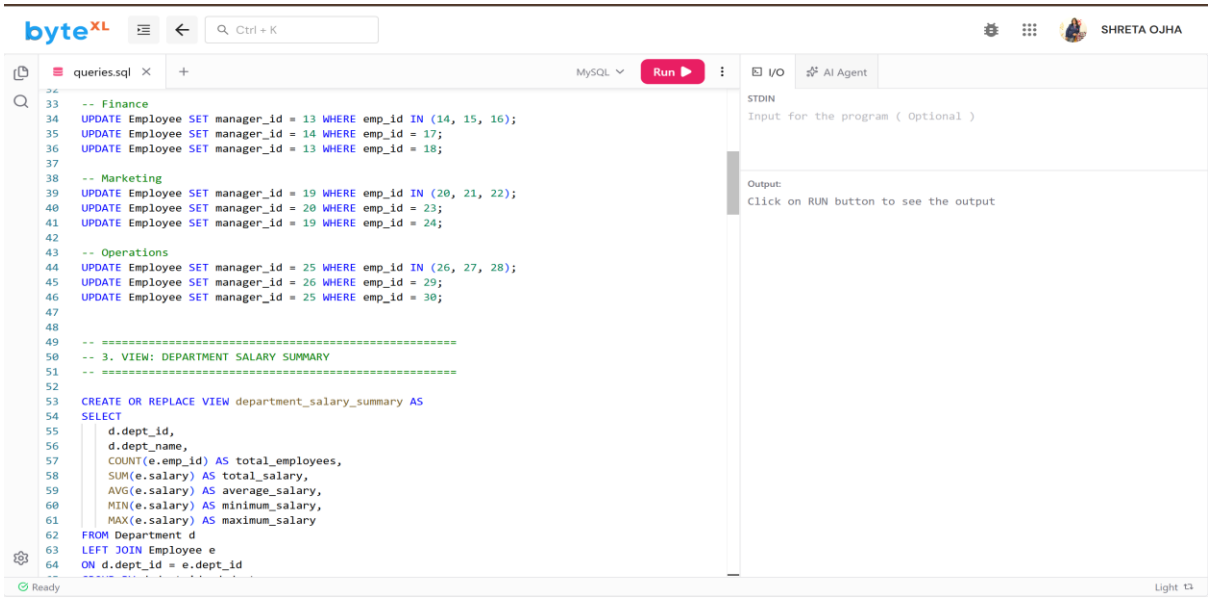

-- ======================================================
-- 2. SET UP EMPLOYEE HIERARCHY
-- ======================================================

-- Top-level employees have NULL manager_id
UPDATE Employee SET manager_id = NULL;

-- IT
UPDATE Employee SET manager_id = 1 WHERE emp_id IN (2, 3, 5);
UPDATE Employee SET manager_id = 2 WHERE emp_id = 4;
UPDATE Employee SET manager_id = 1 WHERE emp_id = 6;

-- HR
UPDATE Employee SET manager_id = 7 WHERE emp_id IN (8, 9, 10);
UPDATE Employee SET manager_id = 8 WHERE emp_id = 11;
UPDATE Employee SET manager_id = 7 WHERE emp_id = 12;

```



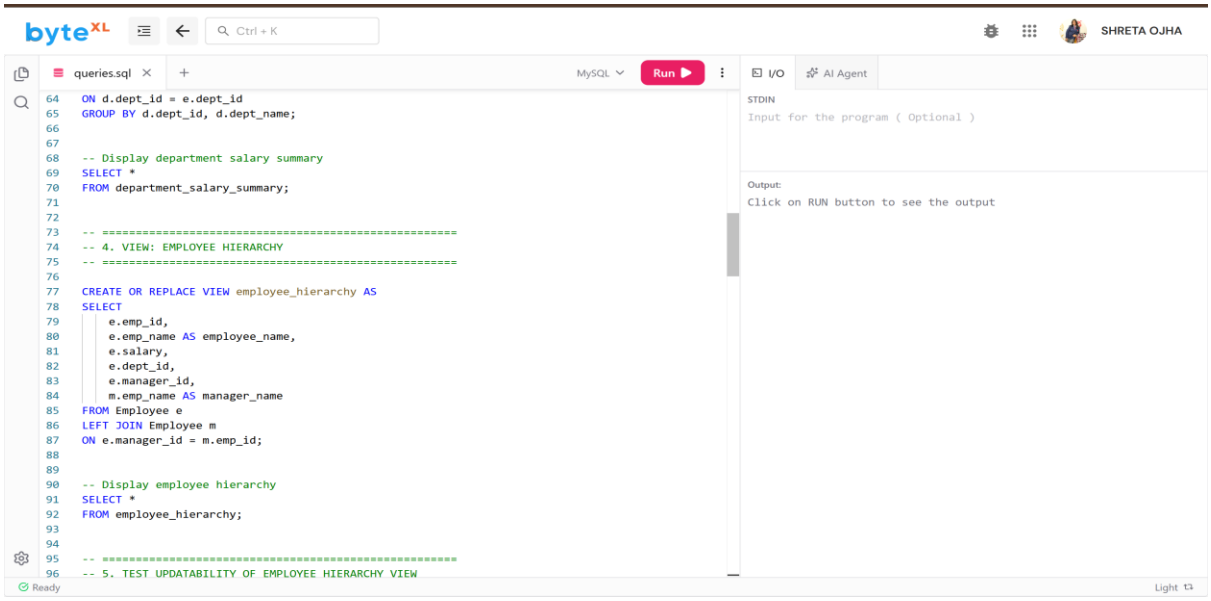
```
-- Finance
UPDATE Employee SET manager_id = 13 WHERE emp_id IN (14, 15, 16);
UPDATE Employee SET manager_id = 14 WHERE emp_id = 17;
UPDATE Employee SET manager_id = 13 WHERE emp_id = 18;

-- Marketing
UPDATE Employee SET manager_id = 19 WHERE emp_id IN (20, 21, 22);
UPDATE Employee SET manager_id = 20 WHERE emp_id = 23;
UPDATE Employee SET manager_id = 19 WHERE emp_id = 24;

-- Operations
UPDATE Employee SET manager_id = 25 WHERE emp_id IN (26, 27, 28);
UPDATE Employee SET manager_id = 26 WHERE emp_id = 29;
UPDATE Employee SET manager_id = 25 WHERE emp_id = 30;


-- ======================================================
-- 3. VIEW: DEPARTMENT SALARY SUMMARY
-- ======================================================

CREATE OR REPLACE VIEW department_salary_summary AS
SELECT
    d.dept_id,
    d.dept_name,
    COUNT(e.emp_id) AS total_employees,
    SUM(e.salary) AS total_salary,
    AVG(e.salary) AS average_salary,
    MIN(e.salary) AS minimum_salary,
    MAX(e.salary) AS maximum_salary
FROM Department d
LEFT JOIN Employee e
ON d.dept_id = e.dept_id
```



```
ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;

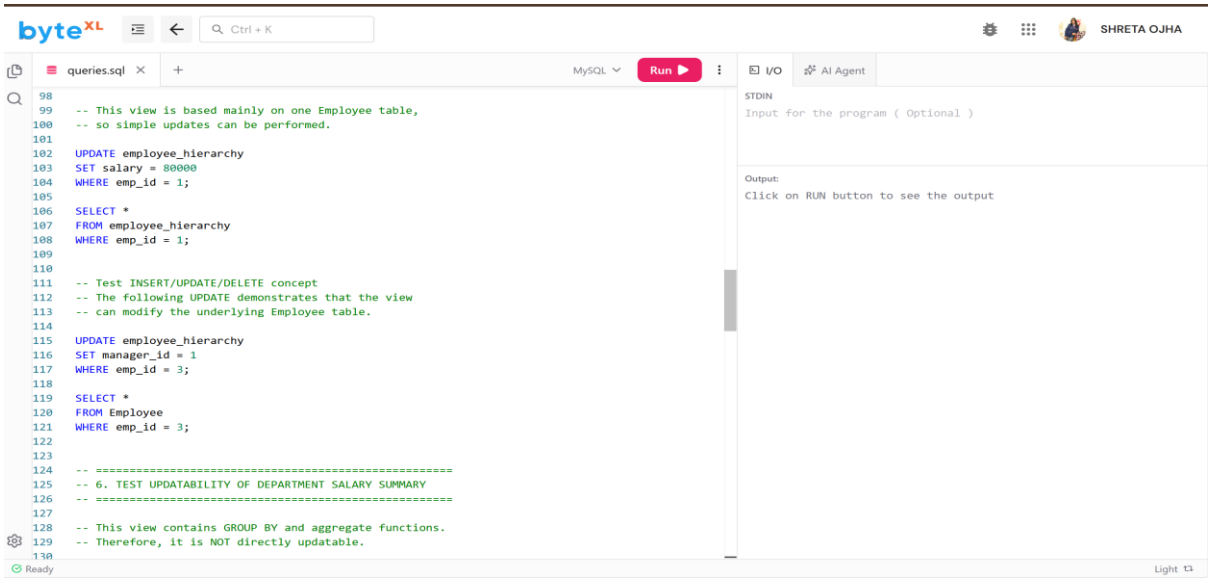

-- Display department salary summary
SELECT *
FROM department_salary_summary;


-- ======================================================
-- 4. VIEW: EMPLOYEE HIERARCHY
-- ======================================================

CREATE OR REPLACE VIEW employee_hierarchy AS
SELECT
    e.emp_id,
    e.emp_name AS employee_name,
    e.salary,
    e.dept_id,
    e.manager_id,
    m.emp_name AS manager_name
FROM Employee e
LEFT JOIN Employee m
ON e.manager_id = m.emp_id;


-- Display employee hierarchy
SELECT *
FROM employee_hierarchy;


-- ======================================================
-- 5. TEST UPDATABILITY OF EMPLOYEE HIERARCHY VIEW
```



```

-- This view is based mainly on one Employee table,
-- so simple updates can be performed.

UPDATE employee_hierarchy
SET salary = 80000
WHERE emp_id = 1;

SELECT *
FROM employee_hierarchy
WHERE emp_id = 1;

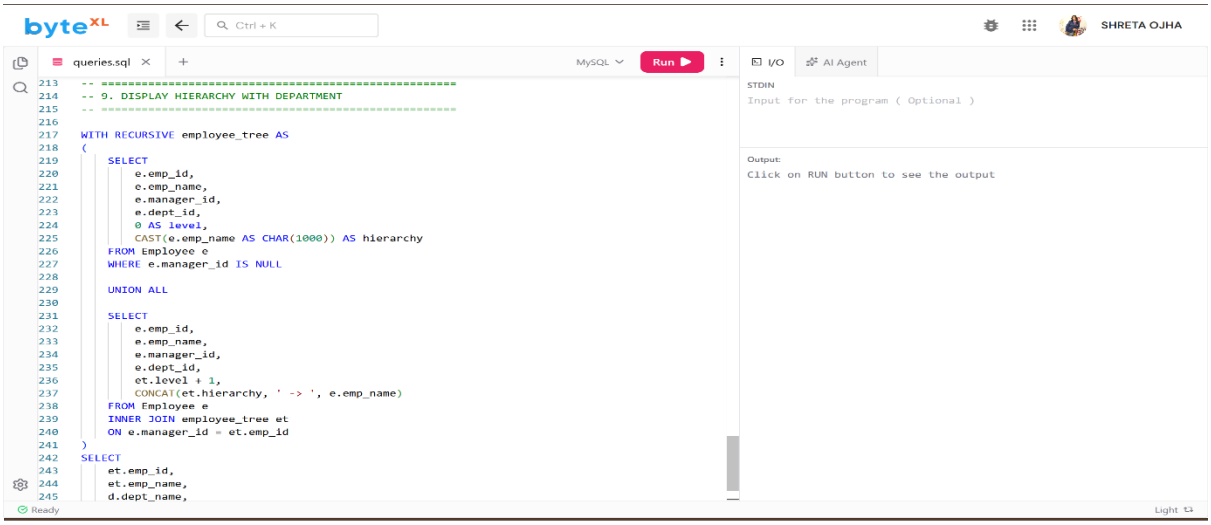

-- Test INSERT/UPDATE/DELETE concept
-- The following UPDATE demonstrates that the view
-- can modify the underlying Employee table.

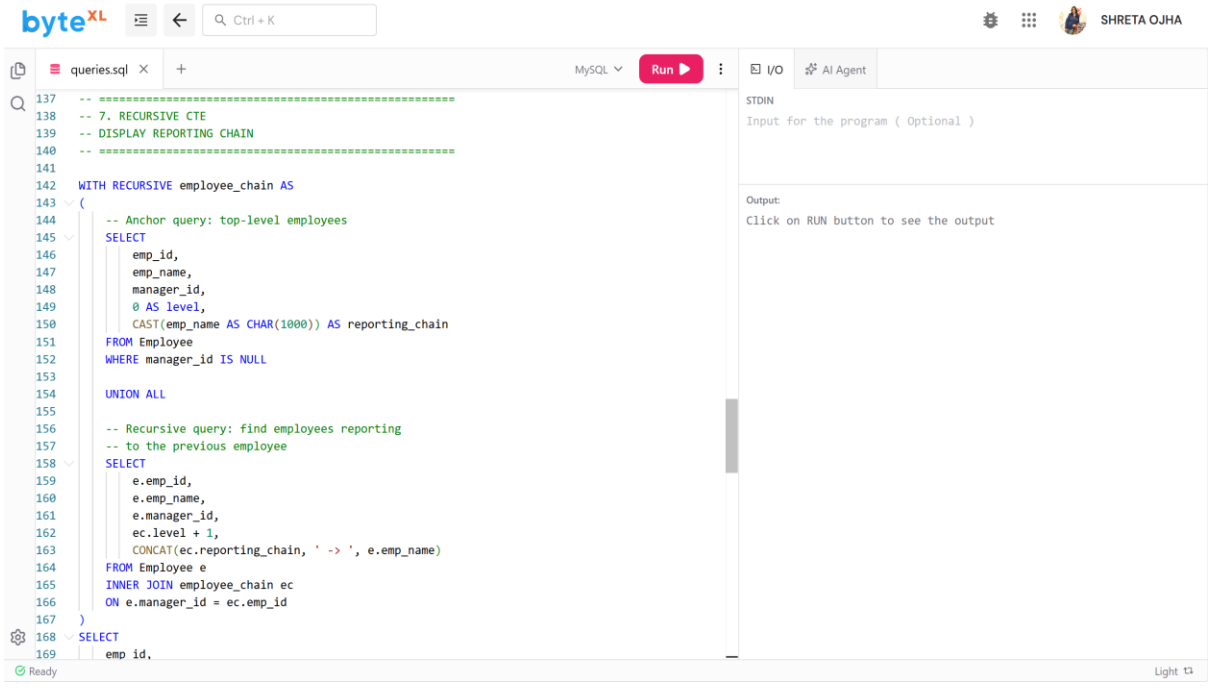
UPDATE employee_hierarchy
SET manager_id = 1
WHERE emp_id = 3;

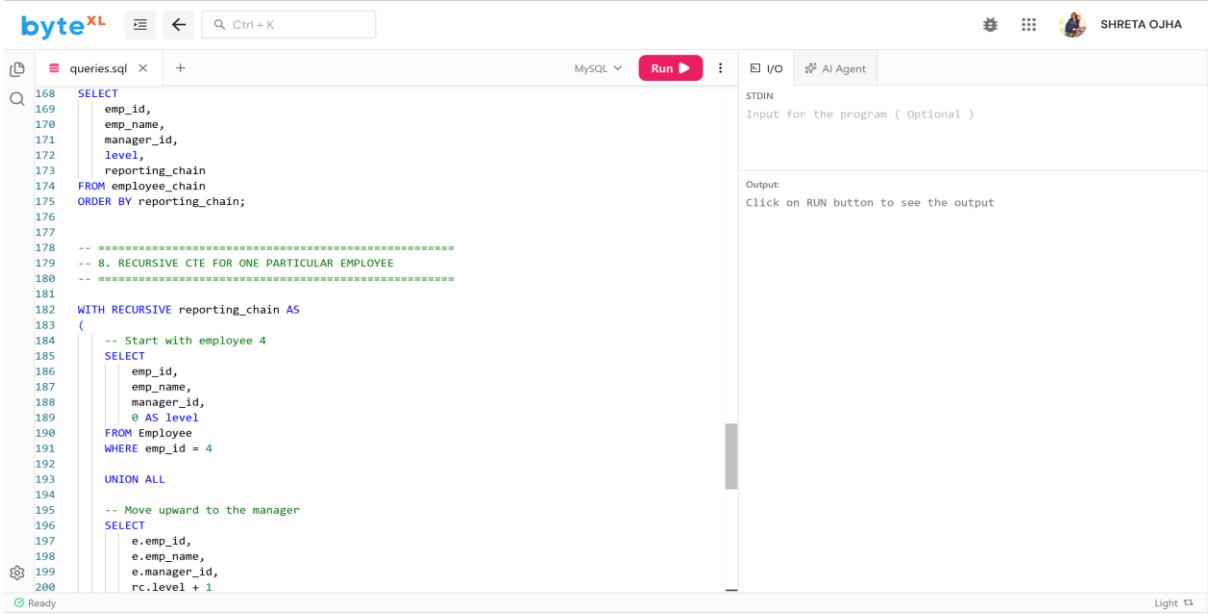
SELECT *
FROM Employee
WHERE emp_id = 3;


-- ======================================================
-- 6. TEST UPDATABILITY OF DEPARTMENT SALARY SUMMARY
-- ======================================================

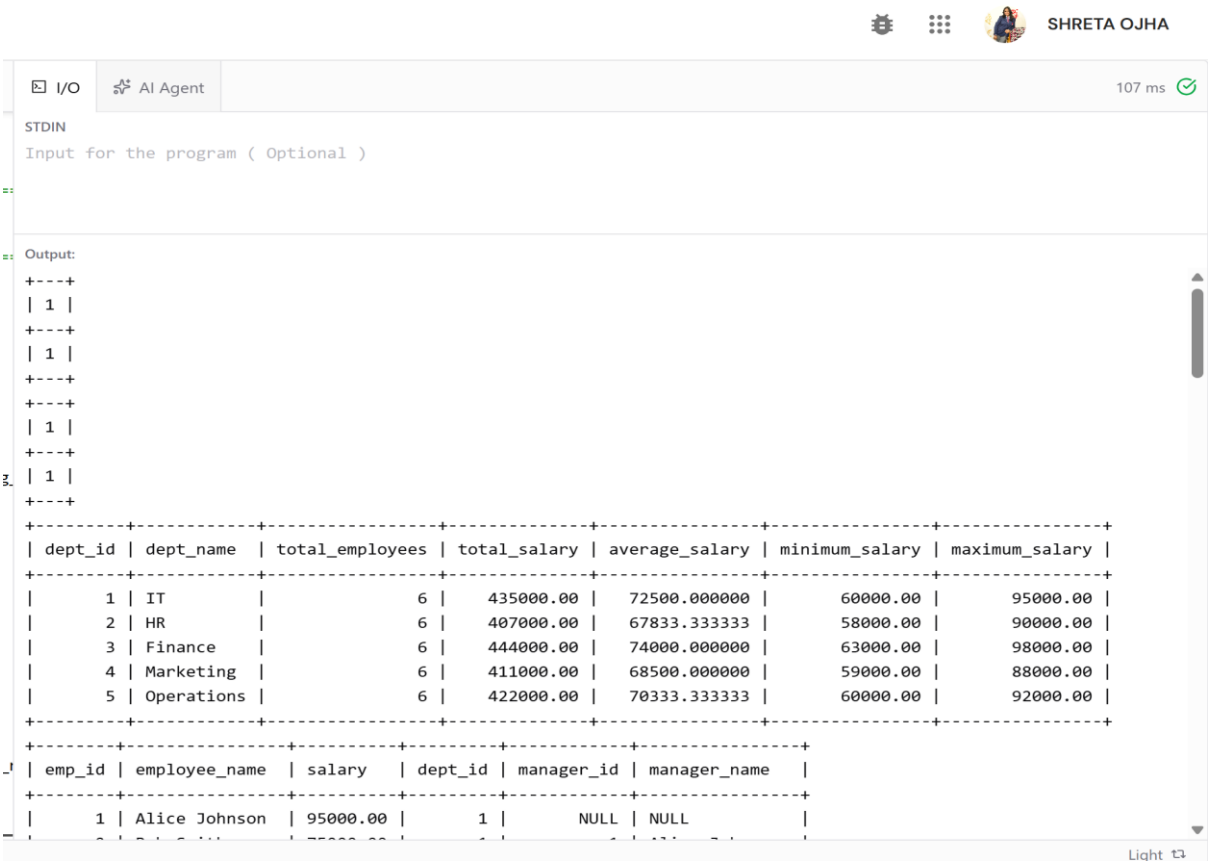
-- This view contains GROUP BY and aggregate functions.
-- Therefore, it is NOT directly updatable.
```


byteXL
Ctrl + K
SHRETA OJHA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
-- ==
-- 9. DISPLAY HIERARCHY WITH DEPARTMENT
-- ==
WITH RECURSIVE employee_tree AS
(
SELECT
e.emp_id,
e.emp_name,
e.manager_id,
e.dept_id,
0 AS level,
CAST(e.emp_name AS CHAR(1000)) AS hierarchy
FROM Employee e
WHERE e.manager_id IS NULL
UNION ALL
SELECT
e.emp_id,
e.emp_name,
e.manager_id,
e.dept_id,
et.level + 1,
CONCAT(et.hierarchy, ' -> ', e.emp_name)
FROM Employee e
INNER JOIN employee_tree et
ON e.manager_id = et.emp_id
)
SELECT
et.emp_id,
et.emp_name,
d.dept_name,
Ready
Light


byteXL
Ctrl + K
SHRETA OJHA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
-- ==
-- 7. RECURSIVE CTE
-- DISPLAY REPORTING CHAIN
-- ==
WITH RECURSIVE employee_chain AS
(
-- Anchor query: top-level employees
SELECT
emp_id,
emp_name,
manager_id,
0 AS level,
CAST(emp_name AS CHAR(1000)) AS reporting_chain
FROM Employee
WHERE manager_id IS NULL
UNION ALL
-- Recursive query: find employees reporting
-- to the previous employee
SELECT
e.emp_id,
e.emp_name,
e.manager_id,
ec.level + 1,
CONCAT(ec.reporting_chain, ' -> ', e.emp_name)
FROM Employee e
INNER JOIN employee_chain ec
ON e.manager_id = ec.emp_id
)
SELECT
emp_id,
Ready
Light


byteXL
Ctrl + K
SHRETA OJHA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
SELECT
emp_id,
emp_name,
manager_id,
level,
reporting_chain
FROM employee_chain
ORDER BY reporting_chain;
-- ==
-- 8. RECURSIVE CTE FOR ONE PARTICULAR EMPLOYEE
-- ==
WITH RECURSIVE reporting_chain AS
(
-- Start with employee 4
SELECT
emp_id,
emp_name,
manager_id,
0 AS level
FROM Employee
WHERE emp_id = 4
UNION ALL
-- Move upward to the manager
SELECT
e.emp_id,
e.emp_name,
e.manager_id,
rc.level + 1
Ready
Light

OUTPUT



SHRETA OJHA

I/O AI Agent 107 ms

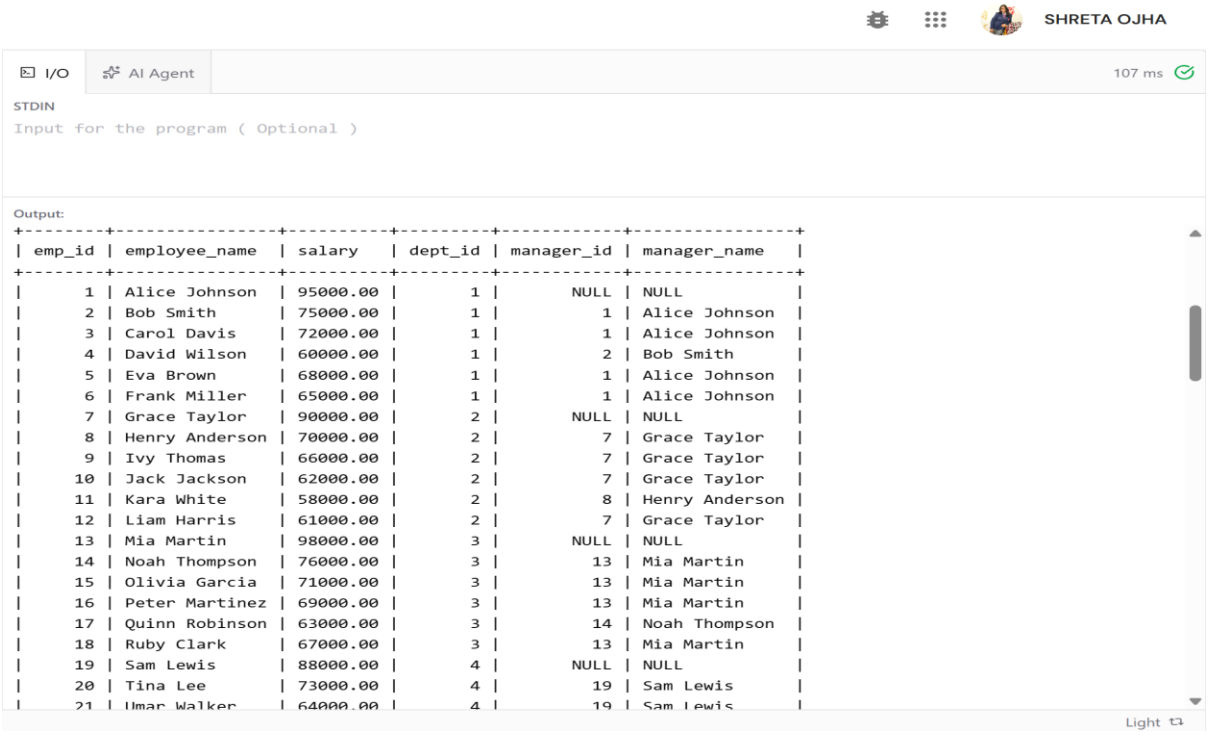
STDIN

Input for the program (Optional)

Output:

```
+---+
| 1 |
+---+
| 1 |
+---+
+---+
| 1 |
+---+
| 1 |
+---+
+---------+------------+-----------------+--------------+----------------+----------------+----------------+
| dept_id | dept_name  | total_employees | total_salary | average_salary | minimum_salary | maximum_salary |
+---------+------------+-----------------+--------------+----------------+----------------+----------------+
|       1 | IT         |               6 |    435000.00 |   72500.000000 |       60000.00 |       95000.00 |
|       2 | HR         |               6 |    407000.00 |   67833.333333 |       58000.00 |       90000.00 |
|       3 | Finance    |               6 |    444000.00 |   74000.000000 |       63000.00 |       98000.00 |
|       4 | Marketing  |               6 |    411000.00 |   68500.000000 |       59000.00 |       88000.00 |
|       5 | Operations |               6 |    422000.00 |   70333.333333 |       60000.00 |       92000.00 |
+---------+------------+-----------------+--------------+----------------+----------------+----------------+
+--------+----------------+----------+---------+------------+----------------+
| emp_id | employee_name  | salary   | dept_id | manager_id | manager_name   |
+--------+----------------+----------+---------+------------+----------------+
|      1 | Alice Johnson  | 95000.00 |       1 |       NULL | NULL           |
```

Light



SHRETA OJHA

I/O AI Agent 107 ms

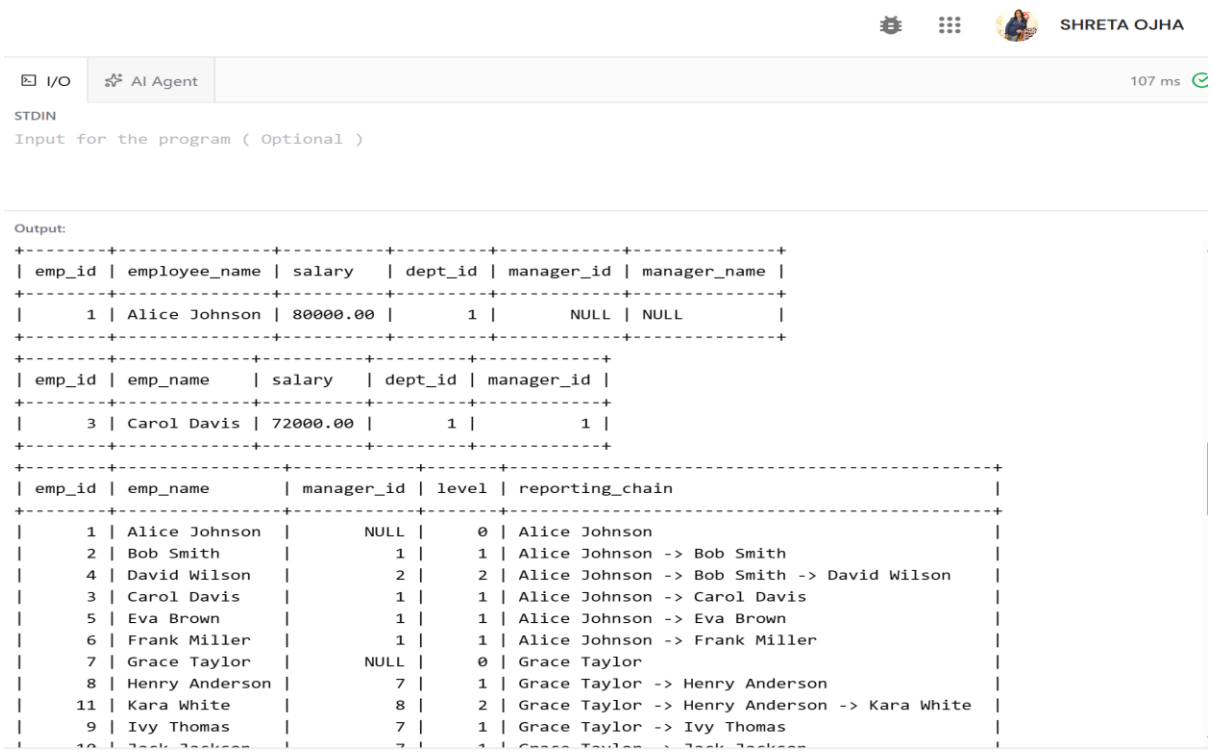
STDIN

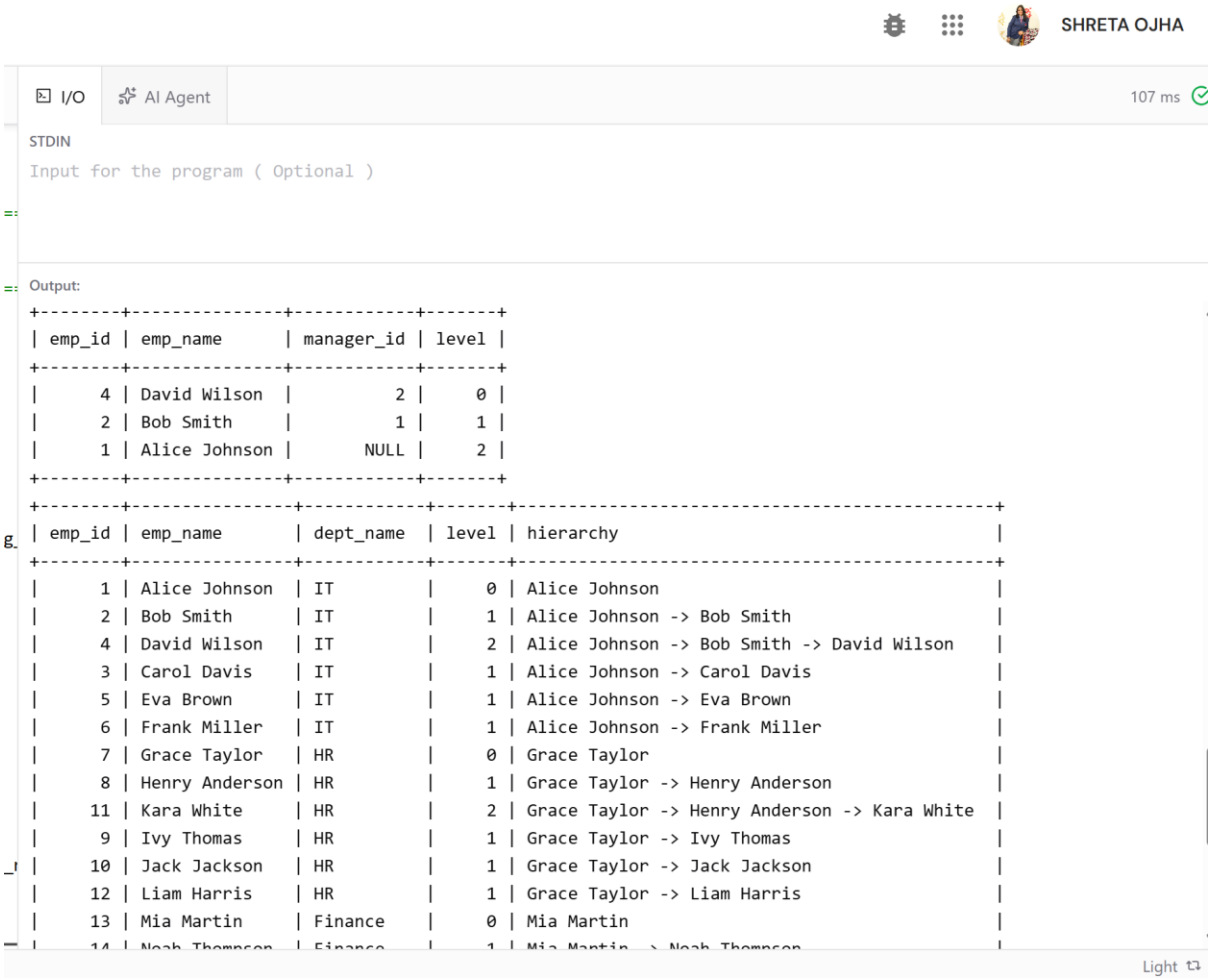
Input for the program (Optional)

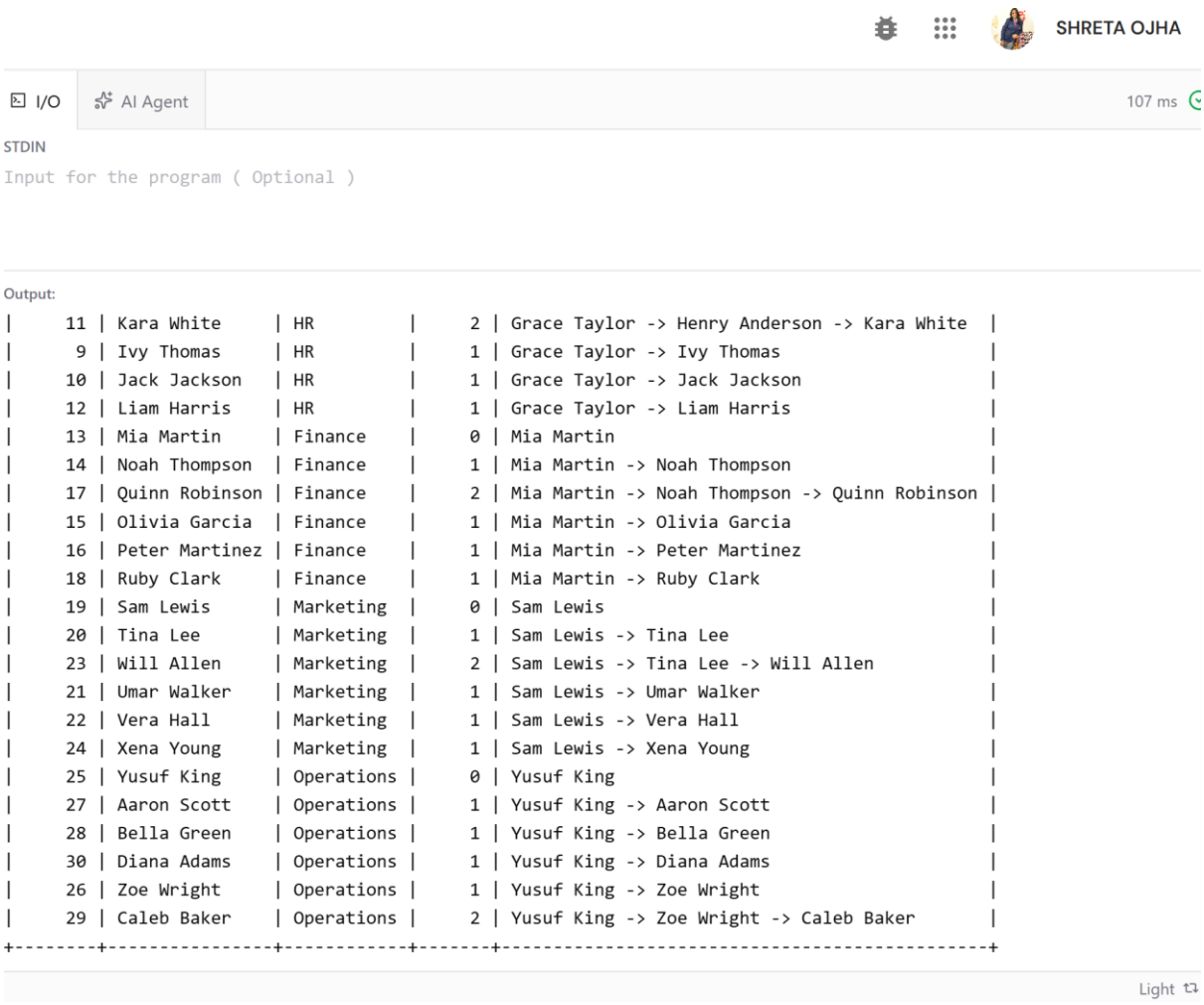
Output:

```
+--------+----------------+----------+---------+------------+----------------+
| emp_id | employee_name  | salary   | dept_id | manager_id | manager_name   |
+--------+----------------+----------+---------+------------+----------------+
|      1 | Alice Johnson  | 95000.00 |       1 |       NULL | NULL           |
|      2 | Bob Smith      | 75000.00 |       1 |          1 | Alice Johnson  |
|      3 | Carol Davis    | 72000.00 |       1 |          1 | Alice Johnson  |
|      4 | David Wilson   | 60000.00 |       1 |          2 | Bob Smith      |
|      5 | Eva Brown      | 68000.00 |       1 |          1 | Alice Johnson  |
|      6 | Frank Miller   | 65000.00 |       1 |          1 | Alice Johnson  |
|      7 | Grace Taylor   | 90000.00 |       2 |       NULL | NULL           |
|      8 | Henry Anderson | 70000.00 |       2 |          7 | Grace Taylor   |
|      9 | Ivy Thomas     | 66000.00 |       2 |          7 | Grace Taylor   |
|     10 | Jack Jackson   | 62000.00 |       2 |          7 | Grace Taylor   |
|     11 | Kara White     | 58000.00 |       2 |          8 | Henry Anderson |
|     12 | Liam Harris    | 61000.00 |       2 |          7 | Grace Taylor   |
|     13 | Mia Martin     | 98000.00 |       3 |       NULL | NULL           |
|     14 | Noah Thompson  | 76000.00 |       3 |         13 | Mia Martin     |
|     15 | Olivia Garcia  | 71000.00 |       3 |         13 | Mia Martin     |
|     16 | Peter Martinez | 69000.00 |       3 |         13 | Mia Martin     |
|     17 | Quinn Robinson | 63000.00 |       3 |         14 | Noah Thompson  |
|     18 | Ruby Clark     | 67000.00 |       3 |         13 | Mia Martin     |
|     19 | Sam Lewis      | 88000.00 |       4 |       NULL | NULL           |
|     20 | Tina Lee       | 73000.00 |       4 |         19 | Sam Lewis      |
|     21 | Umar Walker    | 64000.00 |       4 |         19 | Sam Lewis      |
```

Light


SHRETA OJHA
I/O
AI Agent
107 ms
STDIN
Input for the program (Optional)
Output:
+--------+---------------+----------+---------+------------+--------------+
| emp_id | employee_name | salary | dept_id | manager_id | manager_name |
+--------+---------------+----------+---------+------------+--------------+
| 1 | Alice Johnson | 80000.00 | 1 | NULL | NULL |
+--------+---------------+----------+---------+------------+--------------+
+--------+-------------+----------+---------+------------+
| emp_id | emp_name | salary | dept_id | manager_id |
+--------+-------------+----------+---------+------------+
| 3 | Carol Davis | 72000.00 | 1 | 1 |
+--------+-------------+----------+---------+------------+
+--------+---------------+------------+-------+---+
| emp_id | emp_name | manager_id | level | reporting_chain |
+--------+---------------+------------+-------+---+
1	Alice Johnson	NULL	0	Alice Johnson
2	Bob Smith	1	1	Alice Johnson -> Bob Smith
4	David Wilson	2	2	Alice Johnson -> Bob Smith -> David Wilson
3	Carol Davis	1	1	Alice Johnson -> Carol Davis
5	Eva Brown	1	1	Alice Johnson -> Eva Brown
6	Frank Miller	1	1	Alice Johnson -> Frank Miller
7	Grace Taylor	NULL	0	Grace Taylor
8	Henry Anderson	7	1	Grace Taylor -> Henry Anderson
11	Kara White	8	2	Grace Taylor -> Henry Anderson -> Kara White
9	Ivy Thomas	7	1	Grace Taylor -> Ivy Thomas


SHRETA OJHA
I/O
AI Agent
107 ms
STDIN
Input for the program (Optional)
Output:
+--------+---------------+------------+-------+
| emp_id | emp_name | manager_id | level |
+--------+---------------+------------+-------+
4	David Wilson	2	0
2	Bob Smith	1	1
1	Alice Johnson	NULL	2
+--------+---------------+------------+-------+
+--------+---------------+------------+-------+---+
| emp_id | emp_name | dept_name | level | hierarchy |
+--------+---------------+------------+-------+---+
1	Alice Johnson	IT	0	Alice Johnson
2	Bob Smith	IT	1	Alice Johnson -> Bob Smith
4	David Wilson	IT	2	Alice Johnson -> Bob Smith -> David Wilson
3	Carol Davis	IT	1	Alice Johnson -> Carol Davis
5	Eva Brown	IT	1	Alice Johnson -> Eva Brown
6	Frank Miller	IT	1	Alice Johnson -> Frank Miller
7	Grace Taylor	HR	0	Grace Taylor
8	Henry Anderson	HR	1	Grace Taylor -> Henry Anderson
11	Kara White	HR	2	Grace Taylor -> Henry Anderson -> Kara White
9	Ivy Thomas	HR	1	Grace Taylor -> Ivy Thomas
10	Jack Jackson	HR	1	Grace Taylor -> Jack Jackson
12	Liam Harris	HR	1	Grace Taylor -> Liam Harris
13	Mia Martin	Finance	0	Mia Martin
Light



OBSERVATIONS/OUTCOME

1. A department salary summary view was successfully created using aggregate functions and GROUP BY.
2. An employee hierarchy view was successfully created using a self-join on the Employee table.
3. A ManagerID attribute was added to establish the employee-manager relationship.
4. A simple employee salary view was successfully updated, demonstrating an updatable view.
5. The department salary summary view was found to be non-updatable because it contains aggregate functions and GROUP BY.
6. The employee hierarchy view is based on a self-join and is not treated as a directly updatable view.
7. A recursive CTE was successfully implemented to traverse the employee-manager hierarchy.
8. The recursive CTE displayed the reporting level and complete reporting chain for employees.